LarkFlow 三人协作分工方案（详细版）

> 本文档为团队内部协作计划，**不提交到 Git**（已在 `.gitignore` 中忽略）。
> 最后更新：2026-04-18

0. 阅读指引

- `demo-app/` 是 LarkFlow 运行产物（每次需求都会重写），**不是工作区**；三人都不应该直接修改这里。
- 优化对象是 `LarkFlow/` 框架本身（pipeline / agents / rules / skills）。
- 每位成员的任务都包含：**拥有文件** → **子任务** → **验收标准** → **预估工作量**，直接照做即可。
- 任务之间尽量解耦；必须跨人协作的点集中在 §4「接口契约」，改动需三人会签。

1. 项目当前不足清单（优化依据）

#	位置	问题	影响
1	`pipeline/engine.py:26` `SESSION_STORE`	内存 dict，进程重启丢失，多需求并发无锁	生产级可用性缺失
2	`pipeline/engine.py:48-64` `file_editor`	无沙盒校验（`../../` 可逃逸），缺 `replace` 动作（README 已承认）	安全 & 功能缺失
3	`pipeline/engine.py:43-46` `mock_db`	返回硬编码文本	设计阶段无法做真实 schema 判断
4	`pipeline/engine.py:66-70` `run_bash`	无超时、无 cwd 白名单、无命令黑名单	安全风险，可能卡死
5	`pipeline/engine.py:300-368` `deploy_app`	硬编码 `../../demo-app` 和 Go/Docker 策略	无法对多项目复用
6	`pipeline/engine.py:169-218` `run_agent_loop`	无重试、无超时；全量 `print`，无 token/成本统计	可观测性差，失败即整单挂掉
7	`pipeline/tools_schema.py` vs `agents/tools_definition.md`	工具定义两份副本	极易漂移（已经有 `replace` 的口径不一致）
8	`skills/`	仅 6 个 md；`biz/` 只有 `order.md`；`skill-routing.md` 只有关键词静态映射	知识库覆盖面太窄，命中率低
9	`LarkFlow/Dockerfile`	入口未对齐 FastAPI	容器化运行需先手动修
10	全 `pipeline/`	零单测	重构/回归没有安全网
11	`pipeline/lark_interaction.py`	未见签名校验与 `event_id` 幂等	Webhook 重放即重复触发 pipeline
12	`pipeline/engine.py:240-295` `resume_after_approval`	Phase 2/3/4 串在一个函数里，失败无法从中途断点续跑	任何中间失败都要从 Phase 1 重来

2. 人员分工

👤 A：引擎内核 & 可靠性

> 关键词：状态机、持久化、可观测性、重试、测试

拥有文件

- ✏️ `LarkFlow/pipeline/engine.py`（除 `execute_local_tool` 函数以外）
- 🆕 `LarkFlow/pipeline/persistence.py`
- 🆕 `LarkFlow/pipeline/observability.py`
- 🆕 `LarkFlow/tests/test_engine.py`
- 🆕 `LarkFlow/tests/conftest.py`

子任务

A1. 会话状态持久化（对应问题 #1）

- 新建 `persistence.py`，实现 `SessionStore` 接口：`get(demand_id)` / `save(demand_id, session)` / `delete(demand_id)` / `list_active()`
- 默认实现用 SQLite（文件位置从环境变量读取，默认 `.larkflow/sessions.db`），预留 Redis 实现位
- 将 `engine.py:26` 的全局 `SESSION_STORE: Dict` 全部替换为 `SessionStore` 实例
- 序列化时注意：`session["client"]` 是 SDK 对象不可持久化，须剥离后重建

A2. 阶段状态机显式化（对应问题 #12）

- 在 session 中新增 `phase` 字段：`design | design_pending | coding | testing | reviewing | deploying | done | failed`
- 拆分 `resume_after_approval`，让每个阶段可单独触发与恢复
- 新增 `resume_from_phase(demand_id, phase)` 入口，支持中途失败后从指定阶段续跑

A3. Agent loop 可靠性（对应问题 #6）

- `run_agent_loop` 增加：单轮超时（环境变量 `AGENT_TURN_TIMEOUT`，默认 120s）、最大轮数保护（默认 30 轮）、LLM 调用失败的指数退避重试（与 B 的 `llm_adapter` 的重试解耦，这里是 loop 层）
- 退出条件补齐：超轮数、超时、连续空响应 → 落入 `failed` 态并飞书告警

A4. 结构化日志与指标

- 新建 `observability.py`，提供 `get_logger(demand_id)`（结构化 JSON，输出到 stdout + 文件）
- 埋点：每轮的 `tokens_in / tokens_out / latency_ms / tool_name`，统计到 session 的 `metrics` 字段（在 turn 结束后从 `raw_response.usage` 读取，B 需在 `llm_adapter` 中暴露 usage，见 §4）
- 把 engine.py 里所有 `print(...)` 替换为 logger

A5. 部署层参数化（对应问题 #5）

- 抽 `deploy_app` 中 Go/Docker 专属逻辑为策略（`DeployStrategy` 抽象），内置 `DockerfileGoStrategy`
- `target_dir` 从硬编码的 `../../demo-app` 改为 `session["target_dir"]`，默认仍指向仓库根下的 `demo-app/`，但允许覆盖
- `_classify_deploy_failure` 的分类规则移到策略内部

A6. 单元测试（对应问题 #10）

- 新建 `tests/`（pytest），至少覆盖：`SessionStore` CRUD、阶段状态机转移、`run_agent_loop` 的超时/重试/超轮数、

`deploy_app` 失败分类

- LLM/Lark/subprocess 全部 mock；目标覆盖率 ≥ 70%

验收标准

- ☐ `python pipeline/engine.py` 启动后 kill 进程，重启可恢复未完成的需求
- ☐ Phase 2/3/4 任意一步 LLM 抛错，可通过 `resume_from_phase` 从该阶段重试，不退回 Phase 1
- ☐ `logs/larkflow.jsonl` 中每个需求的 token 总数、总耗时可通过 `jq` 聚合
- ☐ `pytest tests/` 全部通过，覆盖率 ≥ 70%
- ☐ `deploy_app` 可接收非默认 `target_dir` 正常工作

预估工作量

A1 1.5d · A2 1d · A3 0.5d · A4 1d · A5 1d · A6 1.5d ≈ **6.5 人日**

👤 B：工具运行时 & 外部集成

> 关键词：工具沙盒、飞书稳定性、LLM 适配、Dockerfile

拥有文件

- 🆕 `LarkFlow/pipeline/tools_runtime.py`（从 engine.py 抽出的 `execute_local_tool`，**第 0 步重构，暂缓启动**）
- ✏️ `LarkFlow/pipeline/tools_schema.py`
- ✏️ `LarkFlow/pipeline/llm_adapter.py`
- ✏️ `LarkFlow/pipeline/lark_client.py`
- ✏️ `LarkFlow/pipeline/lark_interaction.py`
- ✏️ `LarkFlow/pipeline/utils/lark_doc.py`
- ✏️ `LarkFlow/Dockerfile`

子任务

B0. 第 0 步重构（暂不启动）

- 把 `engine.py:37-72` 的 `execute_local_tool` 原样搬到 `pipeline/tools_runtime.py`，签名保持 `execute(tool_name: str, args: dict, ctx: ToolContext) -> str`
- engine.py 改为 `from pipeline.tools_runtime import execute as execute_local_tool`
- **启动时机**：三人开工前由 B 单独合入一个 PR，之后所有人 rebase
- ⚠️ 当前团队已决定暂缓此步，A 与 B 的工作先在各自文件内完成

B1. `file_editor` 补 `replace` + 沙盒（对应问题 #2、#7）

- `replace` 动作：参数 `{path, old_content, content}`；`old_content` 在文件中出现 0 次或 ≥2 次都要报错
- 路径沙盒：解析 `path` 后必须落在 `WORKSPACE_ROOT`（环境变量，默认仓库根）下；拒绝绝对路径和穿越
- 写操作前检查目录白名单（默认允许 `demo-app/` 下读写，其他目录只读）

B2. `run_bash` 加固（对应问题 #4）

- 默认超时 60s（可被 `args["timeout"]` 覆盖，上限 300s）
- `cwd` 必须在 `WORKSPACE_ROOT` 下
- 命令黑名单：`rm -rf /`、`:(){:|:&};:`、对 `/etc`、`/usr` 的写操作、`sudo`、`curl | sh` 等（用正则粗匹配即可，目的是降事故率，不是安全边界）

- stdout/stderr 超过 100KB 截断

B3. `mock_db` 换真实 schema（对应问题 #3）

- 读 `DATABASE_URL`（SQLite / MySQL），通过 `information_schema` 或 SQLite 的 `sqlite_master` 返回真实表结构
- 未配置 `DATABASE_URL` 时，返回清晰的提示而不是伪造数据
- 保留函数名 `mock_db` 不变（向后兼容），内部实现替换

B4. 工具定义单一真相源（对应问题 #7）

- `tools_schema.py` 保持唯一源
- 新增 `scripts/gen_tools_doc.py`，读取 schema 生成 `agents/tools_definition.md`（顶部加 `<!-- AUTO-GENERATED, DO NOT EDIT -->`）
- 写个 pre-commit hook 或 CI 检查，拦截手改 `tools_definition.md`

B5. 飞书 Webhook 稳定化（对应问题 #11）

- 在 `lark_interaction.py` 新增签名校验中间件（读 `LARK_VERIFICATION_TOKEN` / `LARK_ENCRYPT_KEY`）
- 基于 `header.event_id` 做幂等（用 A 的 `SessionStore` 或单独的 SQLite 表，TTL 24h）
- 合并 `lark_client.py` 与 `lark_interaction.py` 中的消息发送逻辑（现在 `send_lark_card` 与 `send_lark_text` 两套分支各自重复 token 获取）

B6. LLM 适配器增强（为 A4 提供数据）

- `AgentTurn` 结构体新增 `usage: {prompt_tokens, completion_tokens, total_tokens, latency_ms}` 字段
- Anthropic 从 `response.usage`、OpenAI 从 `response.usage` 提取并归一
- `_create_openai_turn` 同样加 `RateLimitError` 外的通用重试（现在只有 RateLimit 有重试）

B7. 修 LarkFlow Dockerfile（对应问题 #9）

- 启动命令改为 `uvicorn pipeline.lark_interaction:app --host 0.0.0.0 --port 8000`
- 按现有 `requirements.txt` 装依赖；镜像基于 `python:3.11-slim`

验收标准

- ☐ 恶意 path `../../../../etc/passwd` 被 `file_editor` 拒绝
- ☐ `run_bash` 跑 `sleep 120` 在 60s 内被 kill 并返回错误
- ☐ 设置 `DATABASE_URL=sqlite:///demo-app/app.db`，`mock_db` 返回真实表结构
- ☐ 同一飞书 `event_id` 推送 3 次，pipeline 只触发 1 次
- ☐ `agents/tools_definition.md` 由脚本生成，内容与 `tools_schema.py` 一致
- ☐ `AgentTurn.usage` 在 Anthropic/OpenAI 两侧都能拿到有效数字
- ☐ `docker build -t larkflow LarkFlow/ && docker run larkflow` 能正常启动 FastAPI

预估工作量

B1 1d · B2 0.5d · B3 1d · B4 0.5d · B5 1.5d · B6 1d · B7 0.5d ≈ **6 人日**

👤 C：Agent 能力 & 规范知识库

> 关键词：Prompt 工程、Skill 扩充、评测集、路由升级

拥有文件

- ✏️ `LarkFlow/agents/phase1_design.md` ~ `phase4_review.md`
- ✏️ `LarkFlow/agents/tools_definition.md`（过渡期；B4 完成后此文件改为自动生成，C 不再手动编辑，改为调整 schema 的 description 字段）
- ✏️ `LarkFlow/rules/flow-rule.md`
- ✏️ `LarkFlow/rules/skill-routing.md`（并新增 YAML 版）
- ✏️ `LarkFlow/skills/**`（含 `biz/`）
- 🆕 `LarkFlow/tests/prompts/fixtures/*.yaml`
- 🆕 `LarkFlow/tests/prompts/eval.py`

子任务

C1. 扩充 skills 库（对应问题 #8）

- 新增横切 skill，每个一份 md，结构保持与现有 `database.md` 一致（都含「何时用、强制规范、反例、示例代码」四段）：
  - `skills/logging.md`：结构化日志字段规范、trace_id
  - `skills/config.md`：配置读取、secrets 不入库
  - `skills/auth.md`：JWT / Session、权限校验位置
  - `skills/rate_limit.md`：限流算法与中间件写法
  - `skills/idempotency.md`：幂等键、防重放
  - `skills/pagination.md`：分页参数、总数查询、游标
- 扩充 `skills/biz/`：参考 `order.md` 再补 `user.md`（注册/登录/风控）、`payment.md`（支付回调幂等、对账）

C2. 路由表结构化（对应问题 #8）

- 新建 `rules/skill-routing.yaml`，格式：

```yaml
- keywords: [redis, cache, 缓存, 分布式锁]
  skill: skills/redis.md
  weight: 1.0
- keywords: [order, 下单, 支付, checkout]
  skill: skills/biz/order.md
  weight: 1.2   # 业务 skill 优先
```

- 保留 `skill-routing.md`（人类可读），在顶部声明以 YAML 为准
- 在 Phase 2 prompt 中要求 agent 先读 YAML 并匹配命中

C3. 四阶段 prompt 评测集（对应问题 #10）

- 新建 `tests/prompts/fixtures/`，每个 yaml 描述一个需求 + 期望行为：

```yaml
name: add_age_field
requirement: 在 users 表增加 age 字段并提供更新接口
expect_phase1_tools: [mock_db, ask_human_approval]
expect_phase2_skills_read: [skills/database.md, skills/http.md]
expect_phase2_files_written_patterns: ["internal/handler/*.go", "db/migrations/*.sql"]
```

- 至少 5 个 fixture：简单 CRUD、Redis 缓存、分页列表、幂等支付回调、并发任务
- `eval.py` 驱动 pipeline（可 mock LLM 或真跑），断言工具调用序列与文件产物命中 pattern

C4. Phase 4 Review → Skills 回灌闭环

- 在 `phase4_review.md` 中要求 Reviewer 把"发现的规范缺口"以结构化块输出（`<skill-feedback>` 标签）
- 新增 `rules/skill-feedback-loop.md`，说明人类/Reviewer 如何把该块转为 `skills/*.md` 的 PR
- 这部分不需要 pipeline 自动化落地，先把流程写清楚即可

C5. Prompt 打磨

- Review 四阶段 prompt 目前均 < 25 行，内容稀薄。参考 Claude/Codex 官方 agent prompt 结构补齐：角色、目标、禁止事项、输出格式、示例
- 每次修改前跑一遍 C3 评测集，改后再跑，确保没有回归

验收标准

- [ ] `skills/` 至少 12 个 md，覆盖以上罗列的领域
- [ ] `rules/skill-routing.yaml` 可被 Phase 2 agent 正确命中（通过 C3 评测集验证）
- [ ] `python tests/prompts/eval.py` 5 个 fixture 全部通过
- [ ] `phase1` ~ `phase4` 每个 prompt 长度 ≥ 50 行且通过评测
- [ ] `skill-feedback-loop.md` 一页内讲清楚闭环步骤

预估工作量

C1 2d · C2 1d · C3 2d · C4 0.5d · C5 1d ≈ **6.5 人日**

---

3. 文件归属速查表

路径	归属	备注
`LarkFlow/pipeline/engine.py`	A	除 `execute_local_tool` 以外
`LarkFlow/pipeline/persistence.py`（新）	A	
`LarkFlow/pipeline/observability.py`（新）	A	
`LarkFlow/pipeline/tools_runtime.py`（新）	B	B0 完成后承接 `execute_local_tool`
`LarkFlow/pipeline/tools_schema.py`	B	工具定义唯一源
`LarkFlow/pipeline/llm_adapter.py`	B	
`LarkFlow/pipeline/lark_client.py`	B	
`LarkFlow/pipeline/lark_interaction.py`	B	
`LarkFlow/pipeline/utils/lark_doc.py`	B	
`LarkFlow/Dockerfile`	B	
`LarkFlow/agents/**`	C	`tools_definition.md` 在 B4 完成后改为自动生成
`LarkFlow/rules/**`	C	
`LarkFlow/skills/**`	C	
`LarkFlow/tests/test_engine.py`（新）、`tests/conftest.py`（新）	A	
`LarkFlow/tests/prompts/**`（新）	C	
`scripts/gen_tools_doc.py`（新）	B	
`README.md` / `CHANGELOG.md` / `.env.example` / `requirements.txt`	集成负责人	三人提需求，统一合并

---

4. 接口契约（跨人协作点，改动需三人会签）

契约 #1：A ↔ B — 工具运行时接口

```python
# pipeline/tools_runtime.py
def execute(tool_name: str, args: dict, ctx: ToolContext) -> str: ...

@dataclass
class ToolContext:
    demand_id: str
    workspace_root: str    # 沙盒根
    target_dir: str        # 当前需求产物目录（默认 demo-app/）
    logger: Any            # A 提供的结构化 logger
```

契约 #2：A ↔ B — AgentTurn.usage

```python
# pipeline/llm_adapter.py
@dataclass
class AgentTurn:
    text_blocks: List[str]
    tool_calls: List[ToolCall]
    finished: bool
    raw_response: Any
    usage: Dict[str, int]   # B6 新增：{prompt_tokens, completion_tokens, total_tokens, latency_ms}
```

契约 #3：B ↔ C — 工具定义唯一源

- `pipeline/tools_schema.py` 为唯一源
- `agents/tools_definition.md` 由 `scripts/gen_tools_doc.py` 生成
- C 要调整工具描述时，改 schema 的 `description` 字段，不改生成的 md

契约 #4：C ↔ A/B — Skill 路由查询接口（可选，C3 若需要程序化匹配时）

```python
# 若 C 需要在 pipeline 里做路由，约定放 pipeline/skill_router.py（B 实现空壳，C 填内容）
def match_skills(text: str) -> List[str]: ...   # 返回命中 skills 文件路径列表
```

---

5. 分支与合并策略

- 长期分支：
  - `feat/engine-*`（A）
  - `feat/tools-*`（B）
  - `feat/agent-*`（C）
- PR 目标：`release/haha_test`
- 合并节奏：每人每完成一个子任务（A1/B1/C1...）开一个小 PR，避免大 PR
- 根目录公共文件（`README.md` / `CHANGELOG.md` / `.env.example` / `requirements.txt`）：改动必须单独开 PR，集成负责人周五统一合并
- 冲突预防：
  1. 三人每日 `git fetch` 并 rebase 自己分支（不做 merge）
  2. 修改接口契约（§4）必须在群里 @ 另两人确认
  3. `engine.py` 与 `tools_runtime.py` 在 B0 完成前有重叠风险，A 动 engine.py 时暂不触碰 `execute_local_tool`；B 动 `execute_local_tool` 时暂不触碰其他函数

---

6. 里程碑建议

周次	A	B	C
W1	A1 持久化	B1 file_editor + B2 run_bash	C1 扩充 skills 前 4 个
W2	A2 状态机 + A3 loop 可靠性	B3 mock_db + B4 工具定义单源	C1 剩余 + C2 路由 YAML
W3	A4 日志指标 + A5 部署参数化	B5 飞书稳定化 + B6 usage	C3 评测集
W4	A6 测试 + 收尾	B7 Dockerfile + B0 重构合入	C4 闭环文档 + C5 prompt 打磨

> 里程碑仅供参考，依据实际进度调整。B0 的第 0 步重构按当前决策放在 W4，届时再评估是否启动。